Expression rewriting as a proxy for bounded model checking

Pepijn Bakker
2024713

Iris Scheepstra
0000000

Konstantinos Tsikouris
0000000

Utrecht University
November 06, 2026

Abstract. Abstract goes here.

1 INTRODUCTION

As part of the Program Semantics and Verification course, we were tasked to write a wlp[1]-based bounded symbolic verification tool for a variation of the GCL language. That is, a tool that can verify whether or not a program satisifies a formal specification, usually in the form of a pre- and post-condition. This particular kind of verifier does this by exploring all possible branches up to a certain bound (hence 'bounded'). Then for each path, it finds the weakest liberal pre-condition, a formula that holds for initial values where that path will terminate in the post-condition, if it terminates. Finally, it checks if the path is valid by solving for the negation of this formula, using SAT or SMT.

Verification of programs, even for simple languages, is challenging due to the computational complexity involved: the number of paths to explore explodes as the search depth increases [1]. Even worse, many of these generated paths cannot logically be reached given any set of initial values (That is, they are *infeasible*). It turns out that the wlp of an infeasible path is always trivially true, so checking these paths is a significant waste of computation [8].

Pruning infeasible paths is therefore an important step in the verification process. However, doing so is costly in and of itself, and will be for naught in cases where the path turns out to be feasible after all. As a result, coming up with a good heuristic for feasibility is an important part of this assignment. Another challenge is the fact that solver queries themselves are quite expensive, especially for longer formulas [6].

For our implementation, we were interested to see how extensive simplification could reduce the overhead of feasibility checking. We also explored a probabilistic method of reducing feasibility checking, based on the exploration depth. During implementation and early testing, we found out that the simplifier alone was able to deem many paths infeasible, and this led to the question of whether the simplifier itself could be a proxy for actual solver queries.

The results showed that in two out of three test programs, the simplifier was able to reason away nearly all infeasible paths. For the third program however, all the pruning was attributed to the solver instead.

Despite this, execution time decreased only by a modest amount. Compared to a control with neither simplifier nor heuristic, the speedups we measured are small: mostly between 1× and 2×, with a maximum of about 2.9× (`pullUp`, $k = 40$). Individual settings are sometimes slower than the control. The combination that lets the simplifier's verdict bypass the solver is the most consistent, with a speedup in nearly all cases on `divByN` and `memberOf` (up to 1.8×), and it never loses more than about 15% on `pullUp`.

[1] weakest liberal precondition

2 RELATED WORK

Symbolic execution and its main obstacle, path explosion, are surveyed by Baldoni et al. [1]. Bounded model checkers such as CBMC [5] unroll the program up to a bound and pass the result to a SAT or SMT solver [3]. We similarily unroll the program but use wlp to reason on each path in a backwards manner. The size of the resulting formulas is a known concern for w(l)p-based verifiers such as Boogie [2], and Leino [6] proposes a formulation that avoids exponential blow-up.

Most symbolic executors check feasibility eagerly. KLEE [4] queries the solver at every branch, and relies on path-search heuristics, including depth-first search, to decide where to explore next. We instead decide whether or not to check at all, using a depth-based probabilistic measure.

KLEE rewrites expressions, splits constraints into independent sets and caches solutions [4] in order to reduce the cost of solver queries. Empc [8] addresses the same problem from the other side, prioritising paths so that fewer infeasible ones are explored. Our simplifier (constant folding, absorption, deduplication and comparison merging) is most adjecent to the expression rewriting KLEE employs. Z3 [7] simplifies formulas itself, so our simplifier mainly saves the cost of calling the solver. This is in line with our measurements: since the solver already performs much of the same rewriting, the additional simplifier only yields a modest speedup, in contrast to the larger gains reported for KLEE's query optimisations [4].

REFERENCES

[1] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3 (May 2018). https://doi.org/10.1145/3182657

[2] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2005. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *Formal Methods for Components and Objects (FMCO) (LNCS)*, 2005. 364–387.

[3] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS)*, 1999. 193–207.

[4] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08)*, December 2008. USENIX Association, San Diego, CA. Retrieved from https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems

[5] Edmund Clarke, Daniel Kroening, and Flavio Lerda. 2004. A Tool for Checking ANSI-C Programs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS)*, 2004. 168–176.

[6] K. Rustan M. Leino. 2005. Efficient weakest preconditions. *Information Processing Letters* 93, 6 (2005), 281–288. https://doi.org/https://doi.org/10.1016/j.ipl.2004.10.015

[7] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS)*, 2008. 337–340.

[8] Shuangjie Yao and Dongdong She. 2025. Empc: Effective Path Prioritization for Symbolic Execution with Path Cover. Retrieved from https://arxiv.org/abs/2505.03555